Noosphere Backend — Database Schema / Data Model Document

Noosphere Backend — Database Schema / Data Model Document

> Generated from a full static scan of `prisma/schema.prisma` (1 datasource, 1 generator, **18 enums**, **~90 models**). This document describes the schema **as defined in code today**, not an idealized target design. Field lists below reproduce the Prisma schema field-for-field (name, type, modifiers) so this document can be used as a standalone data dictionary.

1. Overview

- **ORM**: Prisma 7, `provider = "prisma-client-js"`
- **Database**: PostgreSQL (`provider = "postgresql"`), connected via `@prisma/adapter-pg` driver adapter (see `src/config/prisma.js`)
- **Primary key convention**: `id String @id @default(uuid())` on almost every model. Three models deliberately break this pattern with integer autoincrement IDs: `Payment.id`, `Invoice.id`, `Subscription.paymentId` (FK to `Payment.id`) — see §7.1 for the implications.
- **Timestamp convention**: most models carry `createdAt DateTime @default(now())` and `updatedAt DateTime @updatedAt`. A curated subset (~50 models, tracked in a `createdAtModels` Set in `src/config/prisma.js`) automatically get `orderBy: { createdAt: "desc" }` injected into `findMany` calls application-wide via a Prisma Client extension — callers only need an explicit `orderBy` if they want something other than newest-first.
- **Soft delete convention**: most mutable/business-data models use a boolean `isDeleted` flag rather than physical deletion. A few also carry a separate `isActive`/`active` boolean for enable/disable distinct from delete. Not universal — some models (e.g. `Message`, `PipelineDoneTask`, `AppointmentService`) have no soft-delete flag at all.
- **Multi-tenancy model**: nearly every business table carries a direct or indirect `tenantId String` foreign key to `Tenant`. `Tenant` is the top-level tenant/practice aggregate; almost all domain data (staff, clients, billing config, appointments, forms, clinical reports, payroll) is scoped underneath it. There is no Postgres row-level security — tenant isolation is enforced entirely in the application layer (repositories/services must filter by `tenantId`).
- **Flexible/semi-structured data**: heavy use of `Json`/`Json[]`/`Json?` columns for structured-but-schema-flexible data (see §7.2) — e.g. `Tenant.location`, `BillingPlan.pricePerMonth`/`pricePerYear`, `Target.promptingStrategy`/

masteryCriteria, PipelineItem.doneTasks/sentDocuments, Appointment.recurrence, InvoiceManagement.reminderEmail.

---

2. Enum Reference

Enum	Values
Module	ADMIN, TENANT, CLIENT
SubscriptionStatus	ACTIVE, PAUSED, PENDING, CANCELLED
PlanType	ENTERPRISE, STANDARD
InvoiceStatus	Paid, Upcoming, Due, Overdue
incoiceBillingFrequency *(sic — typo in schema)*	Monthly, Yearly
DomainType	SKILL_ACQUISITION, BEHAVIOR_REDUCTION
DocumentRequestStatus	UPLOADED, PENDING, CANCELLED
WeekDay	MONDAY … SUNDAY
ApprovalStatus	PENDING, APPROVED, REJECTED
ClinicalReportStatus	DRAFT, SUBMITTED, SIGNED, APPROVED, AWAITING_SIGNATURE, CHANGES_REQUESTED
TokenOwnerType	ADMIN, STAFF, CLIENT
ClientFormStatus	PENDING, FILLED
FeatureModule	DASHBOARD, SCHEDULER, CLIENTS, MY_ORGANIZATION, BILLINGS_PAYMENTS, PAYROLL, PROGRAM_LIBRARY, CUSTOM_FORMS, REPORTS, HELP_SUPPORT, SETTINGS, TENANT, BILLING, ISSUE_MANAGEMENT, FEATURE_MANAGEMENT
PaymentSchedule	SALARIED, HOURLY, DAILY
DataAccessLevel	GLOBAL, INDIVIDUAL, TEAM
UserType	ADMIN, TENANT_STAFF, CLIENT
NotificationEntityType	APPOINTMENT, ISSUE, SUBSCRIPTION, INVOICE, PAYMENT, TENANT, PLAN, CLIENT, DOCUMENT_REQUEST, FORM, AUTHORIZATION, CLINICAL_REPORT, LICENSE, TIMESHEET, PAYER, PAYROLL
RescheduleRequestStatus	PENDING, ACCEPTED, REJECTED

Note: `Module` (platform-level: who created a record) and `UserType` (`TENANT_STAFF` instead of `TENANT`) are two separate, overlapping three-way actor enums used in different tables (`Pipeline.module`/`Role.systemModule`/`Auth.module` vs. `Message`/`Notification` sender/receiver typing) — a subtle inconsistency to be aware of when writing cross-cutting queries.

---

3. Top-Level Entity-Relationship Map

The schema is best understood as **13 domain clusters** hanging off a central `Tenant` aggregate, plus platform-level (`Admin`/`Role`) and cross-cutting (`Logs`/`ServerRequest`/`Notification`/`Message`) tables. High-level cardinalities:

```
Admin ──< Tenant (createdBy)                    Admin ──< Tenant
(assignToAdmin / accountOfficer)
Admin ──< Role (createdByAdmin)                 Admin ──<
Department / DepartmentMembers
Admin ──< Pipeline (Admin/Tenant pipelines are shared: Module-
scoped)

Tenant 1─┬──< TenantStaff ──< Role
         ├──< ClientTenant >── Client           (many-to-many
join: a Client can belong to N Tenants)
         ├──< Subscription ──> BillingPlan ──> Payment ──>
PaymentMethod
         ├──< Invoice ──< InvoiceToken           Invoice ──<
Payment
         ├──< Pipeline ──< PipelineStage ──< PipelineItem ──<
{DoneTask, SubmittedDocument, CustomTask, CustomDocument}
         ├──< Domain ──< Program ──< Target ──< {ClientTarget,
SessionData}
         ├──< OrganizationInformation / Licenses / Documents /
DiagnosisCodes / SessionTypes
         ├──< ServiceCodes / RoundingRules / InsuranceType ──<
Payer ──< PayerServiceCodes
         ├──< CompensationTypes / IncomeItems / Deductions /
PayrollCycles ──< PayrollCycleStaffs ──< {IncomeItems, Deductions}
         ├──< Forms ──< FormFields               Forms ──<
FormResponses ──< FormResponseFields
         ├──< ClinicalReportTemplates ──< TemplateSections
         ├──< ClinicalReport ──< {Sections, Histories,
ChangeRequests, Versions}
         ├──< Appointment ──< Session ──< {SessionData,
SessionApproval, TimesheetHistory, SessionUpdateRequest}
         ├──< Teams ──< TeamMembers              Department ──<
DepartmentMembers
         ├──< TenantGeneralSettings /
TenantAdditionalSecurityQuestions / TenantAdminChoices
         ├──< Issue ──< IssueComment
         └──< Logs / ServerRequest (observability, nullable FK
back to Tenant/Admin/Staff/Client)
```

```
ClientTenant 1─┬──< ClientDocuments >── ClientRequestedDocuments
               ├──< ClientAuthorization ──<
ClientAuthorizationService ──> ServiceCodes
               ├──< ClientForm / FormResponses
               ├──< ClientFolder ──< ClientFiles
               ├──< ClinicalReport / ClinicalReportChangeRequest
               ├──< ClientNotificationSettings (1:1)
               └──< ServerRequest

Client 1─┬──< ClientProgram >── Program
         ├──< ClientTarget >── Target ──<
ClientTargetDataCollection
         ├──< PipelineItem
         ├──< Appointment
         └──< Payer (primaryPayer, optional FK)
```

Legend: ──< = one-to-many, >── = many-to-one/inverse, ── between two </> on a join table = many-to-many via an explicit join model.

---

4. Domain Cluster Reference

Each subsection lists every model in the cluster with its full field list (as declared in `schema.prisma`) and relations. `PK` = primary key, `FK→X` = foreign key to model X, `?` = nullable, `[]` = list.

4.1 Platform / Identity (`Admin`, `Role`, `Department`)

`Admin` — platform staff/operators (internal Noosphere team, not tenant staff) - `id` PK, `firstName` (default `"jhc"`), `lastName` (default `"hd"`), `email` (unique), `phoneNumber` (unique), `roleId?` FK→Role, `superAdmin` (bool, default false), `active` (default true), `isDeleted` (default false), `password?`, `administratorPassword?`, `authType?`, `authQuestion?`, `auth2FADone` (default false), `createdAt`, `updatedAt` - Relations: owns/creates `Pipeline`, `PipelineItem`, `Role` (`rolesCreated`), `Department` (created + team-lead), `DepartmentMembers`, `TenantDeactivation`, `Logs`, `BillingPlan` (account manager), `IssueComment`, `Issue` (assigned/logged), `ServerRequest`; is assigned to `Tenant` both as creator (`createdBy`, required) and `AccountOfficer` (optional)

`Role` — RBAC role definition, shared shape for Admin and Tenant-created roles - `id` PK, `name`, `dataAccessLevel` (`DataAccessLevel`), `systemModule` (`Module`), `createdByAdminId?` FK→Admin, `createdByTenantId?` FK→Tenant, `isActive` (default true), `createdAt`, `updatedAt` - Relations: `admins` (Admin[] via `RoleAdmins`), `TenantStaff[]`, `roleModuleAccesses` (`RoleModuleAccess[]`)

`RoleModuleAccess` — per-module permission grants for a Role - `id` PK, `roleId` FK→Role (`onDelete: Cascade`), `module` (`FeatureModule`), `permissions` (`Json[]`)

`Department` / **`DepartmentMembers`** — Admin-side org structure - `Department`: `id` PK, `name`, `createdByAdminId?` FK→Admin, `teamLeadId` FK→Admin (required), `isDeleted` (default false), `isActive` (default true), `createdAt`, `updatedAt` -

`DepartmentMembers`: `id` PK, `departmentId` FK→Department, `adminId` FK→Admin (no soft-delete/unique constraint — duplicate memberships are not prevented at the DB level)

`Auth` — generic secret store keyed by owning user - `id` PK, `userId` (unique — one row per user), `module` (`Module`), `secret`, `createdAt`, `updatedAt` - No FK to Admin/TenantStaff/Client — `userId`/`module` is a loose polymorphic reference resolved in application code, not enforced referentially.

`RefreshTokens` — JWT refresh-token store - `id` PK, `ownerId`, `ownerType` (`TokenOwnerType`), `tokenHash`, `fingerprint`, `expiresAt`, `createdAt`, `used` (default false) - Same polymorphic-owner pattern as `Auth` (no DB-level FK; `ownerId`/`ownerType` resolved in code).

4.2 Tenant & Staff (`Tenant`, `TenantStaff`, licenses/payroll/documents)

`Tenant` — the core multi-tenant aggregate (a customer "practice") - `id` PK, `email`, `phoneNumber`, `active` (default false), `isDeleted` (default false), `subdomain`, `companyName`, `contactPerson`, `companySize`, `organizationType`, `location` (`Json`), `leadSource`, `stage` (default `"UNVERIFIED"`), `website?`, `assignToAdmin?` FK→Admin, `practiceNPI?`, `createdBy` FK→Admin (required), `stripeCustomerId?`, `suspended` (default false), `featuresDisabled` (default false), `suspensionReason?`, `suspendedAt?`, `createdAt`, `updatedAt` - Relations: 30+ inverse relations — effectively every tenant-scoped model in the schema (`TenantStaff`, `ClientTenant`, `Subscription`, `Invoice`, `Payment`, `PaymentMethod`, `BillingPlan`, `Issue`, `Domain`, `OrganizationInformation`/`Documents`/`DiagnosisCodes`/`SessionTypes`/`Licenses`, `Appointment`, `ServiceCodes`, `RoundingRules`, `InsuranceType`, `Payer`, `CompensationTypes`, `IncomeItems`, `Deductions`, `PayrollCycles`, `Forms`, `FormResponses`, `ClinicalReportTemplates`, `ClinicalReport`, `TenantGeneralSettings`, `TenantAdditionalSecurityQuestions`, `Role`, `Teams`, `TenantDeactivation` (1:1), `ServerRequest`, `Logs`, `Pipeline`, `PipelineItem`, `TenantAdminChoices` (1:1))

`TenantDeactivation` — 1:1 audit record of why/when a tenant was deactivated - `id` PK, `tenantId` (unique — enforces 1:1) FK→Tenant, `reason`, `details`, `deactivatedById` FK→Admin, `deactivatedAt` (default now), `reactivationDate?`

`TenantStaff` — a staff member (clinician/admin) within a tenant - `id` PK, `fullName`, `email`, `stage?`, `roleId` FK→Role, `tenantId` FK→Tenant, `dob?`, `gender?`, `npi?`, `address?`, `city?`, `state?`, `zip?`, `country?`, `phoneNumber`, `active` (default true), `isDeleted` (default false), `password?`, `authType?`, `authQuestion?`, `auth2FADone` (default false), `createdAt`, `updatedAt` - Relations: extremely central — `PipelineItem`, `ClientTenant` (as creator + `ClientClinicians` many-to-many), `Teams` (team lead), `TenantStaffLicenses`, `TenantStaffPayroll`, `TenantStaffDocuments`, `Appointment` (`AppointmentClinicians` m2m), `clientDocuments`, `staffAvailabilities`, `sessions` (as approver), `timesheetHistories`, `sessionUpdateRequests`, `clientFiles`, `ClinicalReport` (creator + approver, two named relations), `clinicalReportHistories`, `clinicalReportChangeRequests` (approver), `payrollCycleStaffs`, `teamMembers`, `serverRequests`,

tenantNotificationSettings (1:1) - Note: unlike Admin/Client, email/phoneNumber are **not** marked @unique here (a prior migration, 20260624120000_remove_tenant_staff_phone_number_unique, explicitly dropped the phone-number uniqueness constraint).

TenantNotificationSettings — 1:1 per-staff notification preferences (freeform Json) - id PK, userId (unique) FK→TenantStaff, settings (Json), createdAt, updatedAt

Teams / **TeamMembers** — tenant-side team structure (parallel to Admin's Department) - Teams: id PK, name, tenantId FK→Tenant, teamLeadId? FK→TenantStaff, isDeleted/isActive (defaults false/true wait—isDeleted default false, isActive default true), createdAt, updatedAt - TeamMembers: id PK, teamId FK→Teams, staffId FK→TenantStaff (no unique constraint — duplicate membership rows possible)

TenantStaffLicenses — professional licenses held by staff - id PK, licenseName, licenseNumber, tenantStaffId FK→TenantStaff, issueState, expiryDate, isDeleted (default false)

TenantStaffPayroll — per-staff payroll configuration (distinct from the tenant-wide PayrollCycles system) - id PK, paymentSchedule (PaymentSchedule), ratePerHour (String, not numeric), tenantStaffId FK→TenantStaff, minimumHours?, isDeleted (default false) - Relations: incomeItems/deductions many-to-many via PayrollIncomItems/PayrollDeductions named relations to IncomeItems/Deductions

TenantStaffDocuments — uploaded staff documents - id PK, documentsUrl (Json), tenantStaffId FK→TenantStaff, isDeleted (default false), createdAt

---

4.3 Clients / Patients (Client, ClientTenant, documents, notifications)

Client — a person receiving services; **tenant-independent identity** (see ClientTenant for the tenant relationship) - id PK, firstName, lastName, preferredName?, email (unique), avatarUrl?, phoneNumber (unique), DOB?, gender, primaryPayer? FK→Payer, streetAddress?, city?, state?, country?, zipCode?, caregiverName?, caregiverRelationship?, caregiverPhone?, caregiverEmail?, caregiverStreetAddress?, caregiverCity?, caregiverState?, caregiverCountry?, caregiverZip?, isVerified (default false), isDeleted (default false), createdAt, updatedAt - Relations: PipelineItem, ClientTenant[] (tenantLinks), Logs, ClientProgram, ClientTarget, Appointment

ClientTenant — the join/anchor entity representing a Client's relationship to one Tenant; **this is the identity clientProtect() auth middleware authenticates against**, not Client directly - id PK, clientId FK→Client, tenantId FK→Tenant, createdBy FK→TenantStaff, dbAccess (default false), active (default true), stage, requestAppointment (default true), documentAccess (default true), password, passwordChanged (default false), createdAt, updatedAt - @@unique([clientId, tenantId]) — a client can only have one relationship per tenant - Relations: FormResponses, clientDocuments, clientRequestedDocuments, clinicians (m2m → TenantStaff via

ClientClinicians), forms, clientAuthorizations, clientForms, clientFolders, clinicalReports, clinicalReportChangeRequests, clientFiles, clientNotificationSettings, serverRequests

ClientDocuments — uploaded documents for a client - id PK, tenantClientId FK→ClientTenant, name, documentDetails (Json — includes MIME type/fileUrl, populated by the frontend uncontrolled; see ARCHITECTURE.md §12.4), requestId? FK→ClientRequestedDocuments, isDeleted (required, no default), createdBy? FK→TenantStaff, createdAt, updatedAt

ClientRequestedDocuments — staff-initiated requests for a client to upload something - id PK, tenantClientId FK→ClientTenant, name, description?, allowMultiple (required), status (DocumentRequestStatus, default PENDING), dueDate (required), isDeleted (required), createdAt, updatedAt - Relations: clientDocuments[] (fulfilled uploads against this request)

ClientNotificationSettings — 1:1 per-client notification toggles (13 explicit boolean flags rather than a Json blob — contrast with TenantNotificationSettings.settings: Json) - id PK, tenantClientId (unique) FK→ClientTenant, then 13 Boolean @default(true) flags: appointmentScheduled, appointmentRescheduled, appointmentAboutToStart, appointmentStarted, appointmentCancelled, appointmentCompletedAwaitingFeedback, documentRequested, formShared, authorizationAboutToExpire, authorizationExpired, authorizationUnitsAlmostExhausted, authorizationUnitsExhausted, signatureRequested; createdAt, updatedAt

---

4.4 Sales Pipeline (CRM-style onboarding funnel)

Pipeline — a named pipeline definition, scoped to Module (ADMIN/TENANT/CLIENT context) - id PK, module (Module), name, description?, createdByAdminId? FK→Admin, createdByTenantId? FK→Tenant, isActive (default true), createdAt, updatedAt

PipelineStage — ordered stage within a pipeline - id PK, pipelineId FK→Pipeline, name, colourCode, order (Int), description?, isActive (default true), requiredTasks? (Json), requiredDocuments? (Json), createdAt, updatedAt

PipelineItem — a specific tenant/client's position within a pipeline stage - id PK, tenantId? FK→Tenant, clientId? FK→Client, pipelineStageId FK→PipelineStage, assignToAdmin? FK→Admin, assignToTenantStaff? FK→TenantStaff, doneTasks? (Json), sentDocuments? (Json), isDeleted (default false), createdAt, updatedAt - Note: has **both** a legacy Json blob approach (doneTasks/sentDocuments) **and** normalized child tables (PipelineDoneTask, PipelineSubmittedDocument, PipelineItemCustomTask, PipelineItemCustomDocument) — looks like an in-progress migration from JSON blobs to relational rows (confirmed by migration 20260508115201_add_pipeline_item_custom_tasks_and_documents).

PipelineDoneTask — id PK, pipelineItemId FK→PipelineItem (onDelete: Cascade), taskName, isCompleted (default false), completedAt?, createdAt

PipelineSubmittedDocument — id PK, pipelineItemId FK→PipelineItem (onDelete: Cascade), documentName, fileUrl, isVerified (default false), uploadedAt

PipelineItemCustomTask — id PK, pipelineItemId FK→PipelineItem (onDelete: Cascade), taskName, isRequired (default false), isCompleted (default false), completedAt?, createdAt, updatedAt

PipelineItemCustomDocument — id PK, pipelineItemId FK→PipelineItem (onDelete: Cascade), documentName, isRequired (default false), createdAt, updatedAt

---

4.5 Billing, Subscriptions & Plans

BillingMetadata — 1:1 per-tenant billing profile - id PK, tenantId (unique) FK→Tenant, paymentMethod, billingAddress - Relations: paymentHistory (Transactions[])

Transactions — loosely-modeled transaction log tied to BillingMetadata - id PK, billingMetadataId? FK→BillingMetadata, status - Note: minimal fields (no amount/currency/timestamp) — appears superseded by the more detailed Payment model; likely legacy/unused in current flows.

Subscription — a tenant's active plan subscription - id PK, tenantId FK→Tenant, planId FK→BillingPlan, status (SubscriptionStatus), startDate, billingCycle (String), pauseSchedule?, resumeShedule? *(sic — typo)*, autoRenew? (default true), mailNotification?, endDate (required), paymentId (unique) FK→Payment, createdAt, updatedAt - Two fields are commented out in the schema (lastPaymentStatus, amount) — dead/reserved columns not currently materialized. - Relations: Logs[], invoices (Invoice[])

BillingPlan — a purchasable plan (tenant-specific or catalog-wide) - id PK, planType (PlanType), name, colourCode, description?, pricePerMonth/pricePerYear (Json — structured price objects, not plain numerics), forClient/forStaff (Int seat limits), forStorage (Float), extraFeaturesWithPrice? (Json), extraFeaturesEnabled (default false), active (default true), tenantId? FK→Tenant (assignedTo — plans can be tenant-specific), adminId? FK→Admin (accountManager), createdAt, updatedAt - Relations: features/extraFeatures (m2m → Feature via two named relations), subscriptions, Invoice[]

Feature / **FeatureGroup** — catalog of individually toggleable features attachable to plans - Feature: id PK, name, description?, featureGroupId FK→FeatureGroup, active (default true), managedBy (String — presumably admin id, not FK-enforced), createdAt, updatedAt (commented-out field: applicablePlans Json) - FeatureGroup: id PK, name, active (default true), createdAt, updatedAt

SuperAdminChoices — single/global 2FA & security-question policy toggle (no tenant scoping — presumably a singleton row) - id PK, Authenticator2FA, securityQuestion, setForAll, isEnabled (default true), createdAt, updatedAt

`TenantAdminChoices` — per-tenant override of the same policy shape - `id` PK, `tenantId` (unique) FK→Tenant, same 4 boolean fields as `SuperAdminChoices`, `createdAt`, `updatedAt`

`Payment` — a payment record; **uses an `Int` autoincrement PK**, unlike almost every other model - `id` PK (Int, autoincrement), `tenantId` FK→Tenant, `status`, `transactionId?`, `transactionRef?`, `gateway`, `amount` (Int — implies smallest-currency-unit convention, e.g. cents), `invoiceId` (Int, required) FK→Invoice, `paymentMethodId` FK→PaymentMethod, `createdAt`, `updatedAt` - Relations: `subscription` (1:1 inverse of `Subscription.paymentId`)

`PaymentMethod` — stored payment instrument (tokenized, not raw card data) - `id` PK, `cardType`, `lastFourDigits`, `holderName?`, `tenantId` FK→Tenant, `gatewayToken` (String — Stripe/gateway token reference), `lastUsedAt?`, `isDefault` (default false), `createdAt`, `updatedAt` - Relations: `payment` (`Payment[]`)

4.6 Invoicing

`Invoice` — **uses an `Int` autoincrement PK** (see §7.1) - `id` PK (Int, autoincrement), `tenantId` FK→Tenant, `dueDate`, `status` (`InvoiceStatus`), `planId` FK→BillingPlan, `quantity` (Int), `total` (Int — again smallest-unit convention), `billingFrequency` (`incoiceBillingFrequency`), `subscriptionId?` FK→Subscription, `billingPeriodEnd?`, `emailedAt?`, `stripePaymentIntentId?` (unique), `upcomingEmailedAt?`, `dueEmailedAt?`, `overdueRemindersSent` (`Int[]`, default `[]` — tracks which reminder-day offsets have already fired), `chargeAttemptOffsetsSent` (`Int[]`, default `[]`), `lastChargeAttemptAt?`, `cancellationWarningEmailedAt?`, `createdAt`, `updatedAt` - `@@unique([subscriptionId, billingPeriodEnd])` — prevents duplicate invoices for the same subscription billing period - Relations: `Payment[]`, `invoiceTokens` (`InvoiceToken[]`)

`StripeWebhookEvent` — idempotency ledger for inbound Stripe webhooks - `id` PK, `eventId` (unique — Stripe's event ID, used to de-duplicate retried webhook deliveries), `type`, `createdAt`

`InvoiceToken` — single-use payment-link token tied to an invoice - `id` PK, `invoiceId` FK→Invoice (`onDelete: Cascade`), `tokenHash`, `expiresAt`, `createdAt`, `used` (default false)

`InvoiceManagement` — **singleton** config row driving the `InvoiceManagementJob` cron job (no `tenantId` — one config for the whole platform) - `id` PK, `onPlanPurchase` (Bool), `isDaysBeforeDueDate` (default true), `daysBeforeDueDate` (Int), `upcomingInvoiceHeader`/`Body`, `onDueDate` (Bool), `dueInvoiceHeader`/`Body`, `markOverDue` (Int — days), `unpaidReminderTimesBefore` (Int), `attachInvoiceToReminder` (Bool), `reminderEmail` (`Json` — array of `{ sendOn, header, body }`)

`PaymentAndAccountAccess` — **singleton** config row (no relations found in this scan — likely drives the `PaymentCollectionJob` cron job by analogy to `InvoiceManagement`, though the cron job file wasn't directly inspected for this table) - `id` PK, `chargeOnDueDate`, `chargeLastUsedFirst`, `chargeAlternative` (Bools), `retryBefore`/`retryAfter` (Int), `notifyTenant` (Bool),

notificationEmailHeader/Body, cancelAfter (Int), manualCancel (Bool), suspensionAction, errorMessage, emailAfterAttempts (Int), warningMailHeader/Body, sendOnSubscriptionCancel (Bool), cancelMailHeader/Body

---

4.7 Support / Issue Tracking

Issue — internal support ticket against a tenant - id PK, category, priority?, tenantId FK→Tenant, adminId? FK→Admin (assignedTo), title, adminLoggedById? FK→Admin (loggedBy), status (String, not enum), resolutionDeadline?, attachments? (Json), description, resolutionDescription?, createdAt, updatedAt - Relations: comments (IssueComment[]), Logs[]

IssueComment — id PK, issueId FK→Issue, comment, adminId FK→Admin, createdAt, updatedAt

---

4.8 Clinical Programs (ABA-style skill/behavior tracking)

Domain — top-level clinical domain (e.g. skill acquisition vs. behavior reduction), tenant-scoped - id PK, name, description, tenantId FK→Tenant, domainType (DomainType), isDeleted (default false), createdAt, updatedAt

Program — a program within a domain (or custom, domain-less) - id PK, name, description, domainId? FK→Domain, isCustom (default false), isDeleted (default false), createdAt, updatedAt - Relations: target (Target[]), ClientProgram[]

Target — a specific measurable target/goal within a program — the most field-rich clinical model - id PK, name, description, programId? FK→Program, isCustom (default false), isDeleted (default false), sd (String — "discriminative stimulus"), expectedResponse, teachingProcedure, promptingStrategy (Json), dataCollectionType (String), baselineDataRequired (Bool, required), numberOfTrials?, numberOfTasks? (Int), taskSteps? (Json), masteryMetric (String), masteryCriteria (Json), initialStatus (String), attachment?, notes, createdAt, updatedAt - Relations: ClientTarget[], sessionDatas (SessionData[])

ClientProgram — enrollment of a client in a program - id PK, clientId FK→Client, programId FK→Program, createdAt, updatedAt; @@unique([clientId, programId])

ClientTarget — assignment of a target to a client - id PK, clientId FK→Client, targetId FK→Target, createdAt, updatedAt - Relations: ClientTargetDataCollection[]

ClientTargetDataCollection — raw data-collection entries for a client's target (freeform Json) - id PK, clientTargetId FK→ClientTarget, data (Json), createdAt, updatedAt

---

4.9 Organization Profile & Scheduling Configuration

`OrganizationInformation` — 1:1 tenant profile - `id` PK, `tenantId` (unique) FK→Tenant, `name`, `email`, `phoneNumber`, `website`, `practiceNPI`, `streetAddress`, `city`, `state`, `country`, `zipCode`

`OrganizationLicenses` — `id` PK, `tenantId` FK→Tenant, `licenseName`, `licenseNumber`, `issueState`, `expiryDate`, `isDeleted` (default false)

`OrganizationDocuments` — `id` PK, `tenantId` FK→Tenant, `documentName`, `documentUrl`, `uploadedBy` (String — not FK-typed, likely a staff/admin id resolved in code), `createdAt`, `isDeleted` (default false)

`OrganizationDiagnosisCodes` — `id` PK, `tenantId` FK→Tenant, `code`, `description`, `isActive` (required, no default)

`OrganizationSessionTypes` — tenant-defined appointment/session type catalog - `id` PK, `tenantId` FK→Tenant, `name`, `category`, `staffRolesAllowed` (`Json`), `locationsAllowed` (`Json`), `defaultDuration` (Int), `isActive`, `isBillable` - Relations: `Appointment[]`, `sessionTypeServices` (`SessionTypeService[]`)

`SessionTypeService` — join table linking a session type to a billable `ServiceCodes` entry with modifiers - `id` PK, `serviceCodeId` FK→ServiceCodes, `sessionTypeId` FK→OrganizationSessionTypes, `modifiers` (`Json`), `createdAt`, `updatedAt`

`AvailabilityDays` / **`StaffAvailability`** — staff weekly availability - `StaffAvailability`: `id` PK, `staffId` FK→TenantStaff; relations: `availabilityDays` (`AvailabilityDays[]`) - `AvailabilityDays`: `id` PK, `dayOfWeek` (`WeekDay`), `available` (Bool), `from`/`to` (String — presumably `HH:mm`), `availabilityId` FK→StaffAvailability

---

4.10 Appointments & Sessions

`Appointment` — scheduled/occurred appointment - `id` PK, `clientId` FK→Client, `tenantId` FK→Tenant, `sessionId` FK→OrganizationSessionTypes (naming collision — this FK points to the *session type*, not the `Session` model), `date`/`startTime`/`endTime` (String, not DateTime), `isRecurring` (Bool), `recurrence` (`Json`), `isBillable` (Bool), `serviceLocation`, `requiresTravel` (Bool), `colourCode`, `relatedAppointment?` (self-FK for recurring series linkage), `isCanceled` (default false), `reasonForCancel?`, `reasonForReschedule?`, `rescheduled`/`rescheduleAccepted`/`rescheduleRejected`/`clientRescheduleAccepted`/`clientRescheduleRejected` (all default false), `canceledBy?`, `cancelTime?`, `previousDate?`/`previousStartTime?`/`previousEndTime?` (snapshot of prior schedule on reschedule) - Relations: `clinicians` (m2m → `TenantStaff` via `AppointmentClinicians`), `sessions` (`Session[]`), `appointmentServices` (`AppointmentService[]`), self-referential `relatedTo`/`relatedFrom` (`AppointmentRelation`), `rescheduleRequests` (`AppointmentRescheduleRequest[]`), `rescheduleRequestResult` (1:1 inverse) - Note: no `createdAt`/`updatedAt` on this model (unusual given the convention

elsewhere) — appointment history is instead reconstructed from `previousDate`/`previousStartTime`/`previousEndTime` snapshots and the related `AppointmentRescheduleRequest` audit rows.

`AppointmentRescheduleRequest` — a pending/resolved reschedule proposal - `id` PK, `appointmentId` FK→Appointment, `tenantId`, `clientId` (Strings, not FK-enforced here), `date`/`startTime`/`endTime`, `reasonForReschedule?`, `requestedByType`/`requestedById?` (polymorphic actor), `status` (`RescheduleRequestStatus`, default `PENDING`), `respondedByType?`/`respondedById?`/`respondedAt?`, `newAppointmentId?` (unique) FK→Appointment (the resulting new appointment once accepted), `createdAt`, `updatedAt`

`AppointmentService` — billable service code(s) attached to an appointment - `id` PK, `serviceCodeId` FK→ServiceCodes, `appointmentId` FK→Appointment, `modifiers` (`Json`)

`Session` — an actual service-delivery session tied to an appointment (distinct from `OrganizationSessionTypes`) - `id` PK, `note`, `appointmentId` FK→Appointment, `supervisorApprovalStatus`/`clientApprovalStatus` (`ApprovalStatus`, default `PENDING`), `supervisorId?` FK→TenantStaff, `startTime`/`endTime` (DateTime), `travelStartTime?`/`travelEndTime?`, `createdAt`, `updatedAt` - Relations: `sessionDatas` (`SessionData[]`), `sessionApprovals` (`SessionApproval[]`), `timesheetHistories`, `sessionUpdateRequests`, `authorizationsUsed` (m2m → `ClientAuthorization` via `SessionAuthorizations` — a session can draw down units from multiple authorizations)

`SessionData` — per-target data captured during a session - `id` PK, `sessionId` FK→Session, `data` (`Json`), `targetId` FK→Target, `createdAt`, `updatedAt`

`SessionApproval` — client/guardian sign-off on a session - `id` PK, `sessionId` FK→Session, `confirmDelivery` (Bool), `rateService`/`rateTherapist` (Int, default 0), `feedback?`, `signature` (String — likely a data-URL or S3 key), `createdAt`, `updatedAt`

`TimesheetHistory` — audit trail of timesheet-affecting actions on a session - `id` PK, `sessionId` FK→Session, `action`, `details?`, `createdBy?` FK→TenantStaff, `createdAt`

`SessionUpdateRequest` — a staff request to amend a submitted session - `id` PK, `sessionId` FK→Session, `description`, `requestedBy` FK→TenantStaff, `createdAt`, `updatedAt`

4.11 Tenant Billing Configuration (service codes, payers, insurance)

`ServiceCodes` — billable procedure/service code catalog (e.g. CPT-style codes) - `id` PK, `tenantId` FK→Tenant, `code`, `description`, `modifiers` (`Json`), `isDeleted` (default false), `isActive` (default true) - Relations: `PayerServiceCodes[]`, `sessionTypeServices`, `appointmentServices`, `clientAuthorizationServices`

RoundingRules — billing time-rounding rule definitions - `id` PK, `tenantId` FK→Tenant, `ruleType`, `ruleName`, `description`, `standardUnit?` (Int), `roundingRule?` (`Json`), `isDeleted`/`isActive` defaults - Relations: `PayerServiceCodes[]`

InsuranceType — `id` PK, `tenantId` FK→Tenant, `name`, `description`, `isDeleted`/`isActive` - Relations: `Payer[]`, `clientAuthorizations` (`ClientAuthorization[]`)

Payer — insurance payer/carrier, tenant-scoped - `id` PK, `tenantId` FK→Tenant, `payerName`, `email`, `phone`, `insuranceTypeId` FK→InsuranceType, `tplCode`, `carrierPayerId`, `address`/`city`/`state`/`zip`/`country`, `serviceCodes` (`Json` — a *second*, denormalized representation of service codes alongside the relational `PayerServiceCodes` join table), `isDeleted`/`isActive` - Relations: `PayerServiceCodes[]`, `clientAuthorizations`, `clients` (`Client[]` — inverse of `Client.primaryPayer`)

PayerServiceCodes — rate sheet: a payer's negotiated rate for a specific service code, using a specific rounding rule - `id` PK, `payerId` FK→Payer, `serviceCodeId` FK→ServiceCodes, `unitCurrency` (String), `modifiers` (`Json`), `ratePerUnit` (Int — smallest-unit convention again), `roundingRuleId` FK→RoundingRules, `billable` (Bool, required), `isDeleted`/`isActive`

4.12 Payroll

CompensationTypes — `id` PK, `tenantId` FK→Tenant, `name`, `isDeleted`/`isActive`

IncomeItems / **Deductions** — near-identical shape, tenant-scoped catalog of payroll line-item types - `id` PK, `tenantId` FK→Tenant, `name`, `type` (String), `rate` (`Json`), `isDeleted`/`isActive` - Relations: many-to-many back to `TenantStaffPayroll` (via `PayrollIncomItems`/`PayrollDeductions`)

PayrollCycles — a recurring payroll run definition - `id` PK, `tenantId` FK→Tenant, `name?`, `compensationType?` (`PaymentSchedule`), `interval` (Int), `startDate` (String), `autoRun` (default false), `isDeleted`/`isActive` - Relations: `payrollCycleStaffs` (`PayrollCycleStaffs[]`)

PayrollCycleStaffs — per-staff enrollment/snapshot within a payroll cycle - `id` PK, `payrollCycleId` FK→PayrollCycles, `staffId` FK→TenantStaff, `paymentSchedule` (`PaymentSchedule`), `ratePerHour` (String), `minimumHours?` - Relations: `payrollCycleStaffIncomeItems`/`payrollCycleStaffDeductions` — **note this is a second, cycle-scoped snapshot of income/deduction line items**, structurally parallel to but independent from the tenant-wide `IncomeItems`/`Deductions` catalog (presumably a point-in-time copy so historical payroll runs aren't affected by later catalog edits)

PayrollCycleStaffIncomeItems / **PayrollCycleStaffDeductions** — `id` PK, `payrollCycleStaffId` FK→PayrollCycleStaffs, `name`, `type`, `rate` (`Json`)

4.13 Custom Forms

`Forms` — a form definition, optionally a template or client-specific instance - `id` PK, `tenantId` FK→Tenant, `name`, `isDraft`/`isTemplate`/`isDeleted` (all default false), `tenantClientId?` FK→ClientTenant, `createdAt`, `updatedAt` - Relations: `FormFields[]`, `FormResponses[]`, `clientForms` (`ClientForm[]`)

`FormFields` — a field within a form (rich per-type config via optional `Json` columns) - `id` PK, `formId` FK→Forms, `fieldType` (String), `label`, `placeholder`, `options?` (`Json`), `fileUpload?` (`Json`), `starRating?` (`Json`), `signature?` (`Json`), `isRequired` (default false), `order` (Int) - Relations: `FormResponseFields[]`

`FormResponses` — a single submission of a form - `id` PK, `formId` FK→Forms, `tenantId` FK→Tenant, `submittedBy?` FK→ClientTenant, `submittedAt` (default now) - Relations: `fields` (`FormResponseFields[]`)

`FormResponseFields` — one answered field within a response - `id` PK, `formResponseId` FK→FormResponses, `formFieldId` FK→FormFields, `value` (String — all answer types are stringified, regardless of the field's declared type)

`ClientForm` — tracks whether a specific client has been assigned/completed a specific form (status wrapper distinct from `FormResponses`) - `id` PK, `tenantClientId` FK→ClientTenant, `status` (`ClientFormStatus`, default `PENDING`), `formId` FK→Forms, `createdAt`, `updatedAt`; `@@unique([tenantClientId, formId])`

4.14 Client Authorizations (insurance authorization units)

`ClientAuthorization` — an insurance authorization for a client (a grant of billable units over a date range) - `id` PK, `tenantClientId` FK→ClientTenant, `title`, `authorizationNumber`, `startDate`/`endDate` (String), `payer` FK→Payer, `insuranceType` FK→InsuranceType, `isDeleted`/`isActive` - Relations: `clientAuthorizationServices` (`ClientAuthorizationService[]`), `sessions` (m2m → `Session` via `SessionAuthorizations` — tracks which sessions drew against this authorization)

`ClientAuthorizationService` — per-service-code unit allocation within an authorization - `id` PK, `serviceCodeId` FK→ServiceCodes, `clientAuthorizationId` FK→ClientAuthorization, `modifiers` (`Json`), `units` (Int — total granted), `usedUnit` (Int, default 0 — running consumption counter), `per` (String — unit period, e.g. "week"/"month")

4.15 Files & Folders

`ClientFolder` — a folder for organizing a client's files - `id` PK, `name`, `clientTenantId` FK→ClientTenant, `createdAt`, `updatedAt` - Relations: `clientFiles` (`ClientFiles[]`)

ClientFiles — an uploaded file, optionally organized into a folder - id PK, name, url, size (String), uploadedBy? FK→TenantStaff, clientTenantId FK→ClientTenant, fileType (String), folderId? FK→ClientFolder, createdAt, updatedAt - Note: this is a **separate document-storage subsystem** from ClientDocuments/ClientRequestedDocuments (§4.3) — the two appear to serve different UI surfaces (a general file/folder browser vs. a document-request workflow) without cross-referencing each other in the schema.

4.16 Clinical Reports

ClinicalReportTemplates — a reusable report template per tenant - id PK, tenantId FK→Tenant, title, isActive (default false), isDeleted (default false), createdAt, updatedAt - Relations: reportTemplateSections (ClinicalReportTemplateSection[])

ClinicalReportTemplateSection — id PK, section (String), content (Json), templateId FK→ClinicalReportTemplates

ClinicalReport — an actual report instance for a client, going through a signature/approval workflow - id PK, title, clientTenantId FK→ClientTenant, creatorId FK→TenantStaff (ReportsCreator), approverId? FK→TenantStaff (ReportsApprover), tenantId FK→Tenant, status (ClinicalReportStatus, default DRAFT), isDeleted (default false), tokenVersion (Int, default 0 — likely invalidates outstanding signature/share links on edit), createdAt, updatedAt - Relations: clinicalReportSections, clinicalReportHistories, clinicalReportChangeRequests, clinicalReportVersions

ClinicalReportSection — id PK, section, content (Json), order (Int), clinicalReportId FK→ClinicalReport

ClinicalReportHistory — audit trail of actions on a report - id PK, clinicalReportId FK→ClinicalReport, action, details?, createdBy FK→TenantStaff, createdAt

ClinicalReportChangeRequest — a request (from client or staff reviewer) to revise a report - id PK, description, viewed (default false), clinicalReportId FK→ClinicalReport, clientTenantId? FK→ClientTenant, approverId? FK→TenantStaff (ReportsRequestApprover), createdAt, updatedAt

ClinicalReportVersion — immutable snapshot/export of a report at a point in time - id PK, clinicalReportId FK→ClinicalReport, versionNumber (Int), url (String — presumably an S3-hosted PDF), createdAt

---

4.17 Tenant Settings

TenantGeneralSettings — 1:1, id PK, tenantId (unique) FK→Tenant, dateFormat, timeFormat, currency, createdAt, updatedAt

TenantAdditionalSecurityQuestions — id PK, tenantId FK→Tenant (not unique — a tenant can have multiple questions), question, createdAt, updatedAt

---

4.18 Messaging & Notifications

`Message` — a chat message between any two polymorphic actors - `id` PK, `senderId`/`senderType` (`UserType`), `receiverId`/`receiverType` (`UserType`), `content`, `isRead` (default false), `createdAt` - No FK constraints — sender/receiver identity resolved at the application layer against whichever table `UserType` implies (`ADMIN`→Admin, `TENANT_STAFF`→TenantStaff, `CLIENT`→ClientTenant/Client).

`Notification` — a generic in-app notification, polymorphic subject - `id` PK, `userId`, `userType` (`UserType`), `type` (String), `title`, `content`, `entityType?` (`NotificationEntityType`), `entityId?`, `metadata?` (`Json`), `isRead` (default false), `createdAt` - `@@index([entityType, entityId])` — the only explicit non-PK/unique index in the entire schema, supporting "find all notifications about entity X" lookups.

4.19 Observability

`Logs` — business/audit log entries (see `auditLogger.js` / `audit-request-logger.js` middleware) - `logId` PK (note: non-standard PK name, `logId` rather than `id`), `tenantId?`, `clientId?`, `adminId?` (all optional FKs — polymorphic actor via three nullable relations rather than a discriminated union), `feature?`, `module?` (`Module`), `action`, `reason`, `details?`, `ipAddress?`, `userAgent?`, `outcome?`, `accessedBy?`, `location?`, `subscriptionId?` FK→Subscription, `issueId?` FK→Issue, `createdAt`, `updatedAt`

`ServerRequest` — raw HTTP request telemetry (see `initializeRequestTracking()` in `server.js`) - `id` PK, `tenantId?`/`adminId?`/`tenantStaffId?`/`tenantClientId?` (nullable FKs, `onDelete: SetNull` on all four — request logs survive deletion of the actor), `method`, `endpoint`, `statusCode` (Int), `durationMs` (Int), `ipAddress?`, `userAgent?`, `errorMessage?`, `createdAt`

---

5. Cross-Cutting Relationship Patterns

Pattern	Examples	Notes
Polymorphic actor via multiple nullable FKs	`Logs` (`tenantId`/`clientId`/`adminId`), `ServerRequest` (4 nullable FKs)	No discriminator column — the "actor type" is inferred from which FK is non-null. Requires careful `NULL`-aware querying.
Polymorphic actor via type+id pair (no FK)	`Auth` (`userId`+`module`), `RefreshTokens` (`ownerId`+`ownerType`), `Message`/`Notification` (`*Id`+`*Type` using `UserType`)	Referential integrity is entirely application-

Pattern	Examples	Notes
		enforced; the DB cannot prevent orphaned rows here.
Named multiple relations to the same model	`Admin?Tenant` (`createdBy` vs. `AccountOfficer`), `TenantStaff?ClinicalReport` (`ReportsCreator` vs. `ReportsApprover`), `Appointment?Appointment` (`AppointmentRelation` self-relation), `BillingPlan?Feature` (`PlanFeatures` vs. `ExtraFeatures`)	Standard Prisma pattern for disambiguating multiple FKs between the same two models.
Snapshot/ catalog duplication	`IncomeItems`/`Deductions` (tenant catalog) vs. `PayrollCycleStaffIncomeItems`/ `PayrollCycleStaffDeductions` (per-cycle snapshot); `Payer.serviceCodes` (Json) vs. `PayerServiceCodes` (relational)	Deliberate denormalization for either point-in-time immutability (payroll) or a legacy/quick-access field alongside a normalized join table (payer).
Migration-in-progress duplication	`PipelineItem.doneTasks`/`sentDocuments` (Json) alongside `PipelineDoneTask`/ `PipelineSubmittedDocument` (relational tables added in a later migration)	Both representations currently coexist; application code should be checked for which one is authoritative before relying on either.
1:1 relations enforced via `@unique` on the FK	`TenantDeactivation.tenantId`, `BillingMetadata.tenantId`, `TenantAdminChoices.tenantId`, `OrganizationInformation.tenantId`, `TenantGeneralSettings.tenantId`, `ClientNotificationSettings.tenantClientId`, `TenantNotificationSettings.userId`, `Subscription.paymentId`, `AppointmentRescheduleRequest.newAppointmentId`	Standard Prisma idiom for optional-or-required 1:1.
Composite uniqueness	`ClientTenant (@@unique[clientId, tenantId])`, `ClientProgram (@@unique[clientId, programId])`, `ClientForm (@@unique[tenantClientId, formId])`, `Invoice (@@unique[subscriptionId, billingPeriodEnd])`	Enforces “no duplicate relationship/ period” invariants at the DB level.

Pattern	Examples	Notes
Cascade deletes	`RoleModuleAccess→Role`, `PipelineDoneTask`/ `PipelineSubmittedDocument`/ `PipelineItemCustomTask`/ `PipelineItemCustomDocument→PipelineItem`, `InvoiceToken→Invoice`	Everywhere else, the default Prisma referential action (`Restrict`) applies — deleting a `Tenant`, `Client`, `TenantStaff`, etc. with dependent rows will fail unless the application deletes children first (or soft-deletes instead, which is the dominant pattern per §1).
`SetNull` on delete	`ServerRequest`'s 4 actor FKs	Telemetry intentionally survives actor deletion, just loses the association.

---

6. Prisma Client Extension Behavior (`src/config/prisma.js`)

A global `$extends` hook overrides `findMany` for a fixed allow-list of ~50 models (`createdAtModels`), injecting `orderBy: { createdAt: "desc" }` when the caller didn't specify one. This is **not** a schema-level feature (no default ordering exists in the DB/Prisma schema itself) — it's an app-level convention layered on top. Models with `createdAt` that are **not** in this list (e.g. `Payment`, `TenantStaffLicenses`, `OrganizationLicenses`, `ClinicalReportTemplateSection`… — anything added to the schema after this Set was last updated) will **not** get automatic ordering and must specify `orderBy` explicitly, which is an easy place for the list to silently drift out of sync with the schema.

---

7. Notable Data-Model Risks / Observations

1. **Mixed primary key strategies.** `Payment` and `Invoice` use `Int @default(autoincrement())` while ~88 other models use `String @default(uuid())`. This means: (a) IDs are guessable/enumerable for invoices

and payments specifically (sequential integers), unlike everything else; (b) any code that generically handles "any model's id as a string" (e.g. a shared audit-logging helper) must special-case these two models; (c) `Subscription.paymentId` and `InvoiceToken.invoiceId`/`Payment.invoiceId` are `Int` fields that must stay in sync with this exception.

2. **Heavy reliance on untyped `Json`/`Json[]` columns** for business-critical structured data (pricing objects, permission grants, task/document requirements, mastery criteria, recurrence rules, reminder schedules). This gives flexibility but sacrifices DB-level validation, makes querying/filtering on nested fields harder (JSONB operators or app-side filtering only), and pushes all shape validation into application code (Joi DTOs) with no schema-level backstop if a service bypasses the DTO layer.
3. **Very few explicit indexes.** Only one `@@index` exists in the entire schema (`Notification` on `[entityType, entityId]`). Every other query path relies solely on the implicit indexes Postgres/Prisma creates for `@id` and `@unique` fields. Given nearly every table is filtered by `tenantId` (multi-tenancy) and most list endpoints paginate/sort by `createdAt`, the absence of composite indexes like `(tenantId, createdAt)` on high-traffic tables (`Appointment`, `Invoice`, `ClientDocuments`, `ServerRequest`, `Logs`, `Session`) is a likely performance bottleneck as data volume grows — worth profiling with `EXPLAIN ANALYZE` on the largest tenants.
4. **Loose/polymorphic references without FKs** (`Auth.userId`/`module`, `RefreshTokens.ownerId`/`ownerType`, `Message`/`Notification` sender/receiver, `Issue.status` as free-text instead of an enum, `OrganizationDocuments.uploadedBy` as a bare String) mean the database cannot catch orphaned references or typos in these fields — integrity depends entirely on application code being correct.
5. **Two independent document/file subsystems for clients** (`ClientDocuments`/`ClientRequestedDocuments` vs. `ClientFolder`/`ClientFiles`) with no cross-links — likely fine if they intentionally serve different features, but worth confirming this isn't accidental duplication that should be consolidated.
6. **Naming/typo inconsistencies** that are now baked into the schema and would require a migration to fix: `incoiceBillingFrequency` (enum), `Subscription.resumeShedule` (field), `Logs.logId` (non-standard PK name breaking the `id` convention used everywhere else).
7. **`Appointment` lacks `createdAt`/`updatedAt`**, inconsistent with the rest of the schema, which limits generic auditing/sorting for this otherwise heavily-queried model — the app compensates with manual `previousDate`/`previousStartTime`/`previousEndTime` snapshot fields, but that only captures the *immediately prior* state, not full history (full history lives separately in `AppointmentRescheduleRequest`).
8. **Singleton config tables have no natural uniqueness constraint** (`InvoiceManagement`, `PaymentAndAccountAccess`, `SuperAdminChoices` — all lack any `@unique` field or enforced single-row constraint). Nothing in the schema prevents accidentally creating a second config row, which the cron jobs' `findFirst()`-based reads would then silently ignore (see `InvoiceManagementJob.getConfig()` in ARCHITECTURE.md §10.1) — worth adding an application-level guard or a partial unique index if Postgres version supports it.
9. **Currency/amount fields are plain `Int`** (`Payment.amount`, `Invoice.total`, `PayerServiceCodes.ratePerUnit`) with no explicit currency column at that level (currency is implied contextually, e.g. `PayerServiceCodes.unitCurrency` is a sibling *String* field, and

`BillingPlan.pricePerMonth`/`pricePerYear` bury currency inside a `Json` blob) — inconsistent modeling of money across the schema.